

Автоматы с магазинной памятью

Теория автоматов и формальных языков

Лекция №11

Структура лекции

- Распознавание контекстно-свободных грамматик.
- Автоматы с магазинной памятью.
- Разработка алгоритма синтаксического анализатора с применением МП-автомата.
- Операторные грамматики.
- Алгоритм Дейкстры.
- Матрицы переходов.

Распознавание контекстно-свободных грамматик

- Теория конечных автоматов пригодна только для регулярных грамматик.
- Для распознавания *контекстно-свободных грамматик* существуют более сложные по своей структуре автоматы – **автоматы с магазинной памятью.**

Формальное определение автомата с магазинной памятью

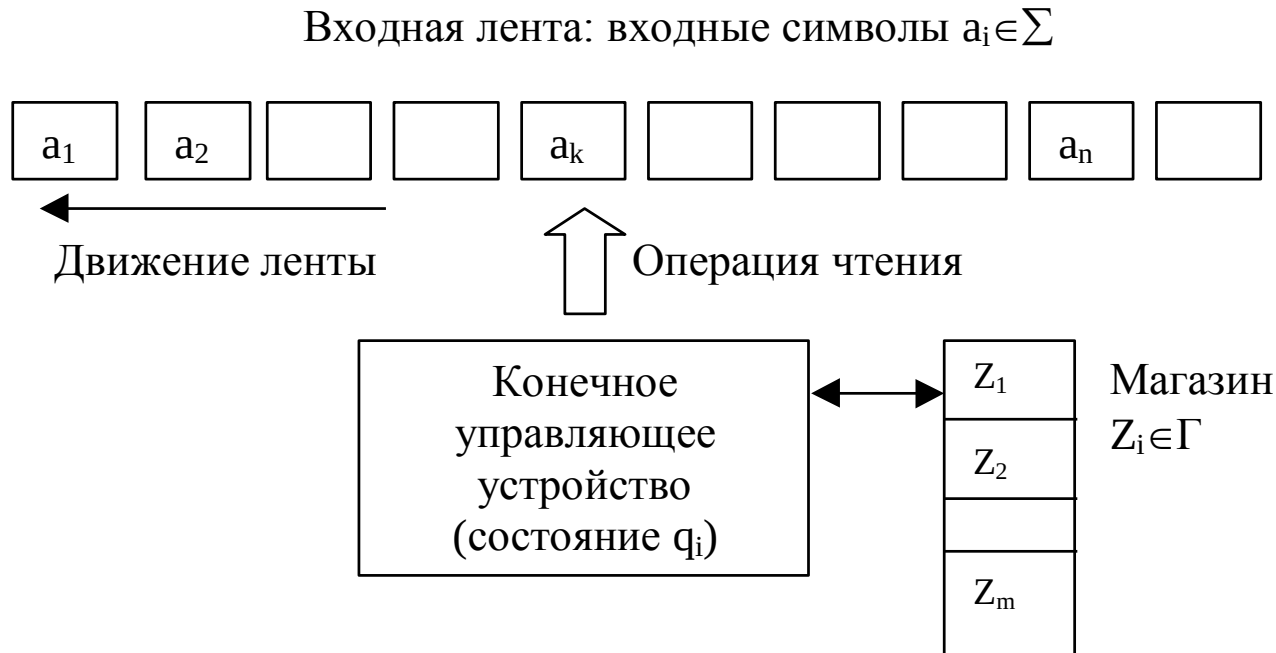
Автомат с магазинной памятью определяется семеркой:

$$P_M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F),$$

где Q – конечное множество состояний автомата;

- Σ – конечный входной алфавит;
- Γ – конечное множество магазинных символов;
- $\delta(q, c_k, z_j)$ – функция переходов;
- $q_0 \in Q$ – начальное состояние автомата;
- $z_0 \in \Gamma$ – символ, находящийся в магазине в начальный момент,
- $F \subseteq Q$ – множество заключительных (допускающих) состояний.

Представление МП-автомата



Конфигурация МП-автомата

Конфигурацией МП-автомата называется тройка $(q, \omega, \alpha) \in Q \times \Sigma^* \times \Gamma^*$, где

- q – текущее состояние устройства;
- ω – неиспользованная часть входной цепочки; первый символ цепочки ω находится под входной головкой; если $\omega = \epsilon$, то считается, что вся входная лента прочитана;
- α – содержимое магазина; самый левый символ цепочки α считается верхним символом магазина; если $\alpha = \epsilon$, то магазин считается пустым.

Конфигурация МП-автомата

Такт работы МП-автомата

- $(q, a\omega, Z\alpha) \rightarrow (q', \omega, \gamma\alpha)$

Начальная конфигурация МП-автомата – это тройка

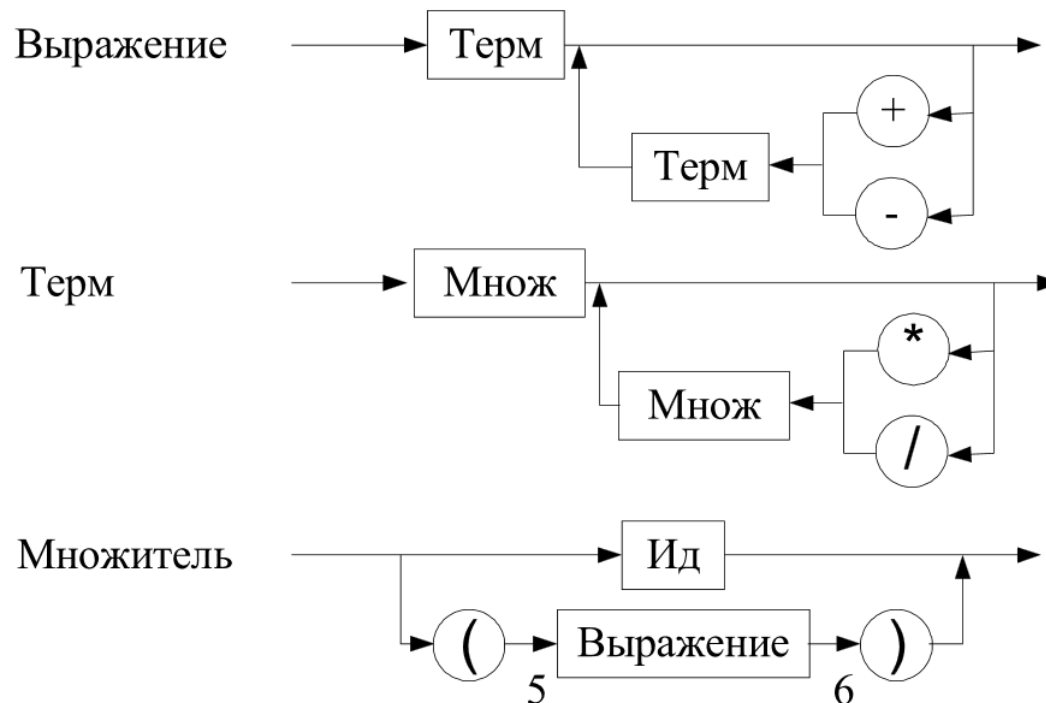
- $(q_0, \omega, Z_0), \omega \in \Sigma^*$

Говорят, что цепочка ω допускается МП-автоматом, если

- $(q_0, \omega, Z_0) \rightarrow^* (q, e, \alpha)$ для некоторых $q \in T$ и $\alpha \in \Gamma^*$

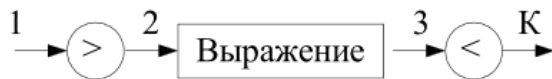
Синтаксический анализатор выражений

- Алфавит языка записи выражений: $\Sigma = \{<\text{Ид}>, +, -, *, /, (,), <, >\}$.
- Грамматика описывается синтаксическими диаграммами, представленными на рисунке:



Синтаксический анализатор выражений

- Получим полную синтаксическую диаграмму, которая состоит из двух частей: основной и рекурсивной, что связано с наличием рекурсивного вложения правил продукции.



Выражение

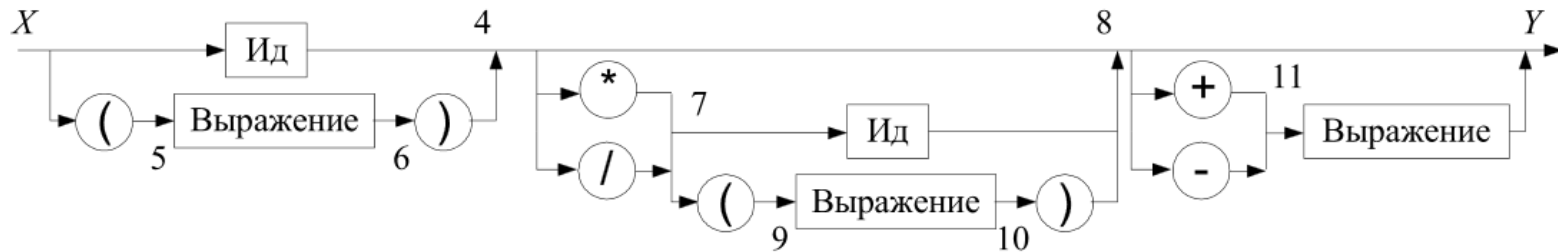


Таблица переходов автомата

	<Ид>	+	-	*	/	(	)	►	◄
1								2	
2	↓ S=3					↓ S=3			
3									К

	<Ид>	+	-	*	/	(	)	►	◄
X	4					5			
4		11	11	7	7		↑ S		↑ S
5	↓ S=6					↓ S=6			
6							4		
7	8					9			
8		11	11				↑ S		↑ S
9	↓ S=10					↓ S=10			
10							8		
11	↓ S=Y					↓ S=Y			
Y							↑ S		↑ S

Условные обозначения:

К – конец разбора,

Е – состояние ошибки (все пустые ячейки должны содержать Е),

↓ Y = <состояние> – рекурсивный вызов распознавателя, справа указан номер состояния после возврата из данного вызова;

↑ Y – возврат из рекурсии, следующее состояние определяется значением Y

Разработка алгоритма, реализующего автомат

При разработке алгоритма используются следующие обозначения:

- **Mag** – стек (магазин) распознающего автомата, элементы, записываемые в стек, – терминальные и нетерминальные символы;
- $\downarrow$ – запись в стек, $\uparrow$ – чтение из стека;
- **q** – текущее состояние;
- **Table** (<Текущее состояние>, <Анализируемый символ>) – элемент таблицы (матрицы) переходов: q – следующее состояние (в том числе « $\downarrow$ » – рекурсивный вызов, « $\uparrow$ » – возврат из рекурсии), S – состояние после возврата из рекурсии, если необходим рекурсивный вызов, или $\emptyset$;
- **String** – исходная строка;
- **Ind** – номер символа исходной строки.

Алгоритм работы автомата

$q := 1$

$Ind := 1$

$Mag := \emptyset$

Цикл-пока $q \neq \text{«E»}$ и $q \neq \text{«K»}$

$q := \text{Table}[q, \text{String}[Ind]].q$

Если $q = \text{'}\downarrow\text{'}$

то $Mag \downarrow \text{Table}[q, \text{String}[Ind]].S$

$q := X$

иначе Если $q = \text{'}\uparrow\text{'}$

то $Mag \uparrow q$

иначе $Ind := Ind + 1$

Все-если

Все-если

Если $q = \text{«K»}$

то «Строка принята»

иначе «Строка отвергнута»

Все-если

Разбор КС-грамматики

- схема СУ-перевода и разбор КС-грамматики с помощью МП-автомата являются *эквивалентными*.
- В частности, если дополнить МП-автомат набором семантических процедур, то МП-автомат, помимо синтаксического разбора, будет уметь формировать и польскую форму записи анализируемой фразы.
- МП-автомат можно считать формальным изложением алгоритма СУ-перевода.

Операторные грамматики

- Метод ОГ основан на понятии приоритета символов анализируемой последовательности, а точнее — приоритета операторов (иногда наряду с термином *операторные грамматики* используется термин *грамматики простого предшествования*).
- Для того, чтобы определить с какой частью цепочки начинать анализ (какое правило выполнять) можно использовать понятие приоритета, основываясь на введенной системе отношений между символами.

Особенности метода разбора операторной грамматики

Достоинства:

- Высокая скорость выполнения и простота метода.
- Непосредственное выполнения выражения.
- Возможно формирование выходных последовательностей в момент вызова процедуры семантической обработки.

Недостатки:

- Отсутствие диагностики.
- Плохое детектирование ошибок.

Алгоритм Дейкстры

- Алгоритм Дейкстры представляет собой менее формализованный вариант метода операторных грамматик.
- Пусть имеется следующая грамматика инфиксных арифметических выражений:
 - $E \rightarrow E + T \mid E - T$
 - $E \rightarrow T$
 - $T \rightarrow T * F \mid T / F$
 - $T \rightarrow F$
 - $F \rightarrow F \wedge P$
 - $F \rightarrow P$
 - $P \rightarrow (E)$
 - $P \rightarrow a$

Алгоритм Дейкстры

Каждой операции ставится в соответствие некоторый *приоритет*, а именно:

+, -	2
*, /	3
^	4

Алгоритм преобразования

Алгоритм преобразования основан на использовании стека и состоит в следующем:

1. Проверяется очередной символ во входной строке.
2. Если это операнд, то он передается в выходную строку.
3. Если это открывающая скобка, то она заносится в стек с приоритетом нуль.
4. Если это операция, то ее приоритет сравнивается с приоритетом операции, находящейся на вершине стека. Если приоритет новой операции больше, то эта новая операция заносится в стек. В противном случае берется операция с вершины стека и помещается в выходную строку, после этого повторяется сравнение с новыми верхними элементами стека до тех пор, пока на вершине стека не окажется операция с приоритетом, меньшим, чем у текущей операции, или пока стек не станет пустым. После этого текущая операция заносится в стек.
5. Если текущий символ во входной строке является закрывающей скобкой, то операции из стека последовательно переносятся в выходную строку до тех пор, пока на вершине стека не появится открывающая скобка; эта открывающая скобка отбрасывается.
6. Если выражение закончилось, то из стека последовательно переносятся в выходную строку все оставшиеся в нем операции.

Анализ программ

- Перейдем от анализа арифметических выражений к более сложным объектам – программам.
- Пусть есть язык, включающий в себя оператор присваивания, а также условный и циклический операторы.
- Грамматика такого языка в самом упрощенном виде может выглядеть так:

прогр ::= инстр

инстр ::= инстр; инстр

инстр ::= перем := выр

инстр ::= if выр then инстр else инстр end

инстр ::= while выр do инстр end

выр ::= выр $\pm$ перем / перем

перем ::= i

Пример программы, анализируемой грамматикой

```
d := 10;  
a := b+c-d;  
if a then d := 1 else  
    while d do  
        e := e-1  
    end  
end
```

Матрицы переходов (*таблицы решений*)

$\langle \text{прог} \rangle ::= \langle \text{инстр} \rangle$

$\langle \text{инстр} \rangle ::= \text{IF } \langle \text{выр} \rangle \text{ THEN } \langle \text{инстр} \rangle$

$\langle \text{инстр} \rangle ::= \langle \text{перем} \rangle := \langle \text{выр} \rangle$

$\langle \text{выр} \rangle ::= \langle \text{выр} \rangle + \langle \text{перем} \rangle \mid \langle \text{перем} \rangle$

$\langle \text{перем} \rangle ::= i$

Для реализации матрицы переходов используются **стек**, **переменная**, хранящая текущий считываемый символ, и **переменная**, в которой будет храниться значение последней приведенной формы.

Матрица переходов

- Стек необходим для хранения правых частей грамматики, заканчивающихся терминалом. Эта часть называется *головой правила*.
- Строки матрицы переходов соответствуют головам правых частей правил, которые могут появиться в стеке, а столбцы соотносятся с терминальными символами. Элементами матрицы будут номера или адреса подпрограмм.

	#	IF	THEN	:=	+	i
#	5	1	0	6	0	1
IF	0	0	9	0	3	1
IF <выр> THEN	7	1	0	6	0	1
<перем> :=	8	0	0	0	3	1
<выр> +	4	0	4	0	4	1
i	2	0	2	2	2	0

Структура стека

Элементами стека являются не отдельные символы, а цепочки символов – головы правых частей правил.

Последовательность:

IF <выр> THEN IF <выр> THEN <перем> :=

в стеке будет представлена следующим образом:

<перем> :=
IF <выр> THEN
IF <выр> THEN
#

Алгоритм работы распознавателя

- На каждом шаге работы верхний элемент стека соответствует некоторой строке матрицы.
- По входному терминальному символу (это – столбец) определяется элемент матрицы, являющийся номером подпрограммы, которую нужно выполнить.
- Эта подпрограмма произведет необходимую редукцию или занесет R в стек и будет сканировать следующий исходный символ.
- При написании подпрограмм будем считать, что функция **PUSH(R)** помещает в стек символ R ;
функция **SCAN()** помещает очередной символ из входной цепочки в R ;
функция **POP()** извлекает из стека верхний символ (предполагается, что он уже не нужен).

Алгоритм составления подпрограмм

- Схема рассуждений при формировании подпрограмм выглядит так: сначала выписываем содержимое верхнего элемента стека. Далее записываем значение считанного терминала и смотрим, что может находиться между ними
- каждый элемент матрицы является номером подпрограммы, которая либо заносит входной символ в стек, либо выполняет редукцию.

Подпрограммы, полученные на основе матрицы переходов

1. *IF* $U \neq \text{'<выр>'}$ *THEN* *ERROR*(1);
 PUSH(*R*);
 SCAN().
2. *IF* $U \neq \text{'<выр>'}$ *THEN* *ERROR*(2);
 POP();
 $U := \text{'<перем>'}$.
3. *IF* $U \neq \text{'<выр>'}$ *AND* $U \neq \text{'<перем>'}$ *THEN* *ERROR*(3);
 PUSH('<выр>+')
 $U := \text{'<+>'}$;
 SCAN().
4. *IF* $U \neq \text{'<перем>'}$ *THEN* *ERROR*(4);
 POP();
 $U := \text{'<выр>'}$.
5. *IF* $U \neq \text{'<прог>'}$ *AND* $U \neq \text{'<инстр>'}$ *THEN* *ERROR*(5);
 STOP().

Подпрограммы, полученные на основе матрицы переходов

6. *IF U≠'<перем>' THEN ERROR(6);
 PUSH('<перем>:=')
 U := '';
 SCAN().*
7. *IF U≠'<инстр>' THEN ERROR(7);
 POP();
 U := '<инстр>'.*
8. *IF U≠'<выр>' AND U≠'<перем>' THEN ERROR(8);
 POP();
 U := '<инстр>'.*
9. *IF U≠'<выр>' AND U≠'<перем>' THEN ERROR(9);
 PUSH('IF <выр> THEN')
 U := '';
 SCAN().*
0. *ERROR(10);
 STOP().*

Особенности разбора при использовании матрицы переходов

- Все подпрограммы должны быть уникальными, так как *каждая* подпрограмма выдает свою собственную диагностику ошибок.
- Процедура синтеза осуществляется тогда, когда происходит редукция формы, т.е. тогда, когда срабатывает какое-либо правило грамматики и переменная U получает значение.

Достоинства и недостатки

- Способ матриц переходов – это очень **быстрый метод обработки**, т.к. в нем полностью исключается поиск.
- Очень эффективная **нейтрализация ошибок**: каждый ноль в матрице соответствует определенной синтаксической ошибке.
- По имеющейся грамматике языка матрица переходов может быть сформирована *автоматически*.
- Недостаток метода матрицы переходов заключается в том, что для более сложной грамматики получается матрица очень большого размера, что влечет массу проблем особенно в том случае, если пытаться сформировать ее вручную.

Задание по лабораторной работе №3

Провести синтаксический анализ программы (не менее 30 строк) на выбранном языке программирования.

- Написать структуру формального МП-автомата для разбора предложений выбранной программы:
 - Формальное определение языка;
 - Синтаксическая диаграмма правил разбора;
 - Таблица переходов.
- Провести разбор программы на бумаге:
 - Составить таблицы идентификаторов, символов, выражений (лб. 1)
 - Привести не менее трех примеров разбора строки программы.
 - Сформировать польскую постфиксную запись для разобранных выражений.

Задание 2 – компьютерная обработка

- Написать программу, основанную на формальной грамматике, позволяющую осуществить синтаксический разбор другой программы (с ограниченным набором операторов – условия, циклы).
 - Программа должна формировать набор таблиц имен, идентификаторов, выражений.
 - Программа должна уметь выявлять синтаксические ошибки в коде.
 - Программа должна уметь выполнять арифметические выражения, встречаемые в компилируемой программе.

Задание 3 - дополнительное

- В соответствии с разработанной формальной грамматикой написать программу, которая может сама написать простую программу из используемых операторов, идентификаторов, выражений по заданной последовательности правил (аналог – слайды 19-20).

Проверочная работа

1. Типы программ, включающие элементы трансляторов.
2. Назовите элементы компиляторов.
3. Перечислите этапы процесса компиляции.
4. Перечислите формы записи выражений и их особенности.
5. Определение алфавита языка.
6. Определение формального языка.
7. Определение и структура формальной грамматики.
8. Форма Бэкса-Науэра.
9. Грамматический разбор – цель, методы.
10. Восходящий и нисходящий грамматический разбор – особенности, отличия, преимущества.
11. Назовите основные типы грамматик по классификации Хомского. Какие из них используются в языках программирования.
12. Лексический анализ – определение, методы.
13. Синтаксический анализ – определение, методы.
14. Определение конечного автомата. Особенности при синтаксическом разборе.
15. Способы описания конечного автомата.
16. Определите автомат с магазинной памятью. В чем особенность его использования при грамматическом разборе?
17. Как построить автомат с магазинной памятью по синтаксическим диаграммам?